

SMART WAREHOUSE TRACKER

A File-Based Spring Boot Web Application for Warehouse Inventory Management

Project Report

Submitted as part of the VITYarthi "Build Your Own Project" evaluation

Course: [Insert Course Name]

Student Name: [Insert Name] | Registration No.: [Insert Reg. No.]

Date: [Insert Submission Date]

2. Introduction

Warehouse and store-room inventory is, in many small and mid-sized operations, still tracked using spreadsheets or handwritten registers. This works at small scale but breaks down as the number of SKUs grows: there is no dependable record of who removed or added stock and when, no automatic warning before an item runs out, and searching through a large list becomes slow and error-prone.

Smart Warehouse Tracker is a Java-based web application, built with Spring Boot, that digitizes this process. It gives a warehouse operator a browser-based dashboard to manage inventory items, record every stock movement, search and filter the catalog, and receive automatic low-stock alerts — all without needing to install or administer a database server, since all data is persisted to simple JSON files on disk.

This report documents the objective, requirements, architecture, design, and implementation of the system, along with the testing approach, challenges encountered, and possible future enhancements.

3. Problem Statement

Small warehouse operators need a simple, dependency-free way to track inventory quantities, know when stock is running low, and keep an auditable history of stock movements — without the overhead of setting up and maintaining a full database server or a heavyweight enterprise inventory system.

Smart Warehouse Tracker addresses this by providing a self-contained Spring Boot web application where:

- Every item has its own low-stock threshold (since a bulk hardware item and an expensive spare part have very different reorder points).
- Every stock addition or removal is logged with a timestamp and reason, forming a complete audit trail.
- All data is stored as human-readable JSON files, so the application can be run anywhere Java is available with zero external setup.

4. Functional Requirements

The system implements three major functional modules, each with a clear input/output structure and a logical workflow for the operator:

4.1 Inventory Management Module (CRUD)

- Input: item SKU, name, category, location, quantity, low-stock threshold, unit price (via a web form).
- Processing: validates required fields and non-negative numeric values; assigns a unique ID and timestamps on creation.
- Output: item persisted to items.json; inventory list page reflects the change immediately.

4.2 Stock Movement Module (Stock In / Stock Out)

- Input: item ID, quantity to add or remove, and an optional reason (e.g. 'New shipment', 'Order #123 fulfilled').
- Processing: for stock-out, the system checks that sufficient quantity is available before proceeding; on success, the item's quantity is updated and a StockMovement record is appended to the audit log.
- Output: updated item quantity; a new entry visible on the Movement History page.

4.3 Search, Reporting & Dashboard Module

- Input: free-text search query, category filter, location filter, and a 'low stock only' toggle.
- Processing: filters the in-memory item list accordingly; aggregates totals (SKU count, total units, total value, low-stock count) for the dashboard.

- Output: filtered inventory table; downloadable CSV export; dashboard summary cards and low-stock alert list.

5. Non-Functional Requirements

Requirement	How It Is Addressed
Performance	All items are cached in memory (backed by the JSON file) so reads are O(1) list scans with no disk I/O per request; writes persist the full file synchronously, which is acceptable at small-warehouse data volumes.
Reliability	A ReentrantReadWriteLock guards the in-memory cache from concurrent read/write corruption across simultaneous requests.
Usability	A clean, responsive Thymeleaf UI with clear navigation (Dashboard / Inventory / Movement History), inline validation messages, and confirm-before-delete prompts.
Maintainability	Layered architecture (Controller -> Service -> Repository) with each class having a single responsibility, enabling changes to storage (e.g. swapping JSON for a real database) without touching controllers.
Error Handling	A centralized @ControllerAdvice (GlobalExceptionHandler) converts domain exceptions (ItemNotFoundException, InsufficientStockException) into friendly error pages instead of raw stack traces.
Logging / Monitoring	Spring Boot's logging is configured to write to logs/warehouse-tracker.log with DEBUG level for the application package, providing traceability of requests and errors.
Resource Efficiency	No external database process, connection pool, or ORM overhead is required; the entire application runs as a single JVM process.
Scalability (documented limitation)	The file-based design is appropriate for a single warehouse with a modest item count; Section 14 describes how the Repository layer could be swapped for a real database as data volume grows.

6. System Architecture

The application follows a classic layered MVC architecture. The Controller layer handles HTTP requests and renders Thymeleaf views; the Service layer contains all business logic (validation, low-stock rules, stock movement rules); and the Repository layer abstracts persistence, reading and writing the JSON files on disk. This separation means the storage mechanism (flat files today) could be replaced by a real database later with no changes to the Controller or Service layers.

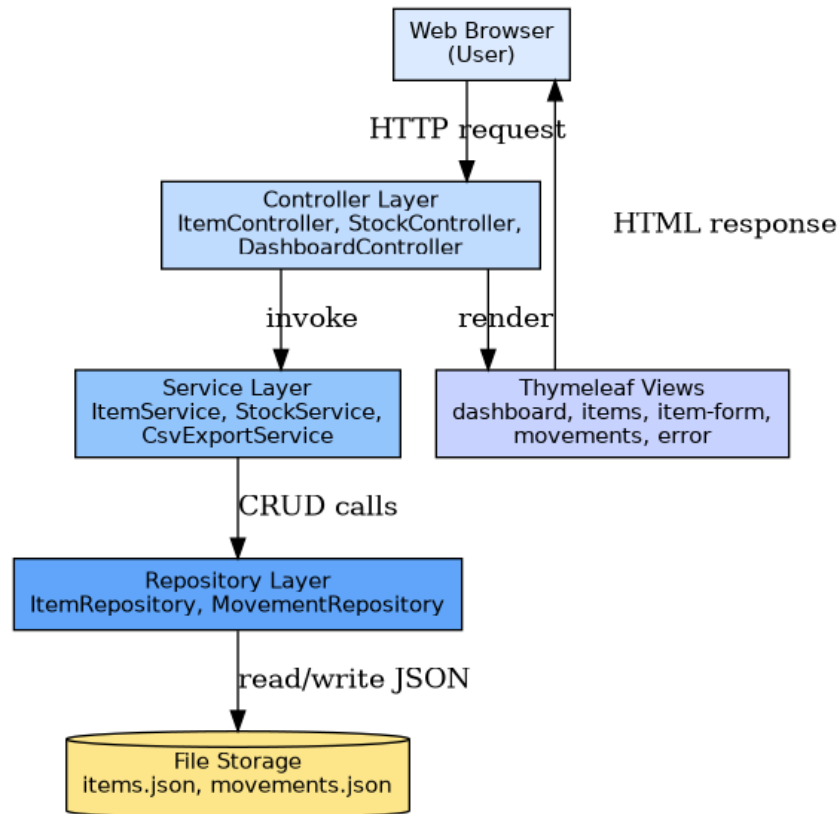


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

The single actor — the Warehouse Operator — interacts with nine primary use cases covering inventory CRUD, stock movement, search, reporting, and the dashboard.

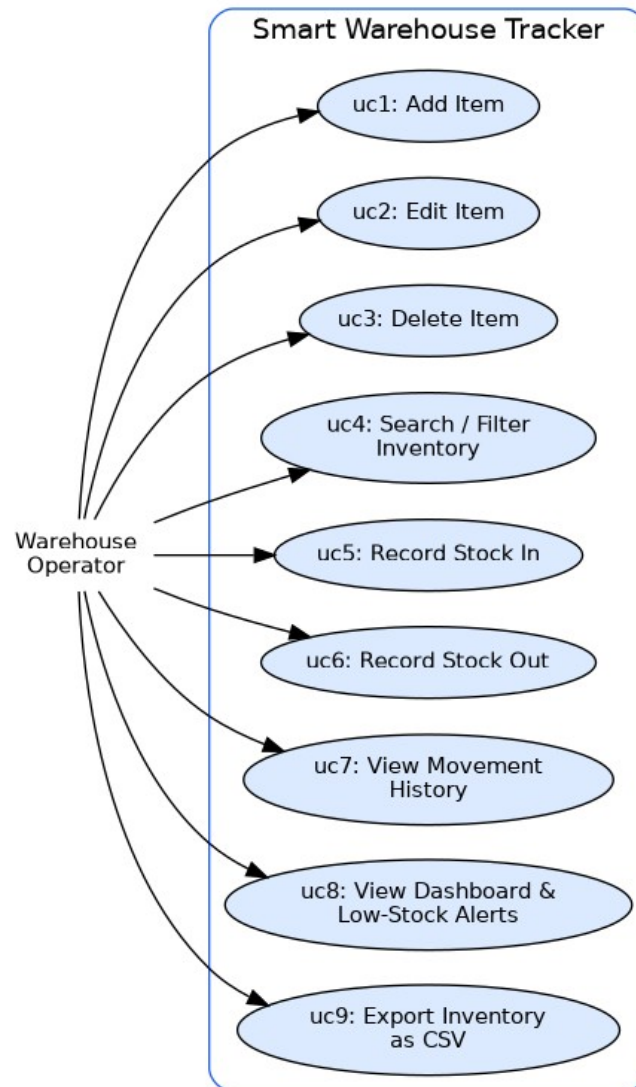


Figure 2: Use Case Diagram

7.2 Workflow / Process Flow Diagram

This diagram traces the operator's decision path from the dashboard through to whichever module they choose, including the validation and stock-availability checks that occur before any change is persisted.

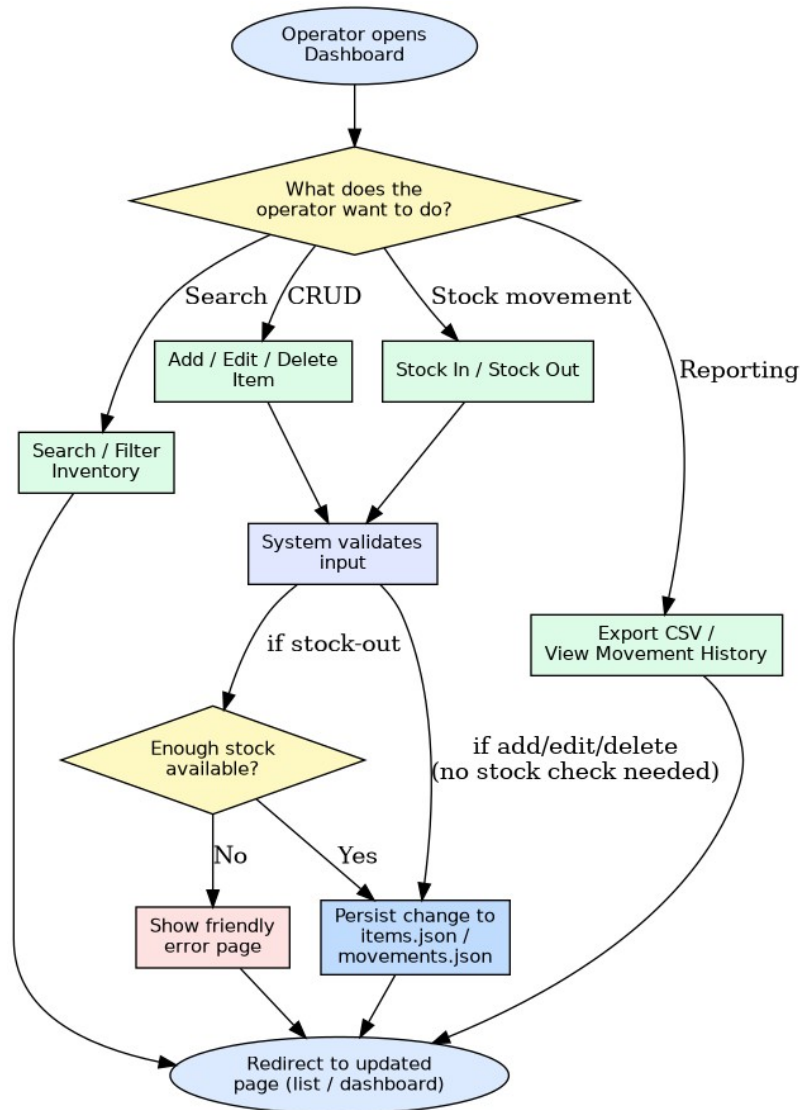


Figure 3: Process Flow / Workflow Diagram

7.3 Sequence Diagram — Stock-Out Flow

This sequence diagram illustrates the most business-critical flow: removing stock. Note the branch where insufficient stock results in an exception being raised and a friendly error page shown, rather than the quantity going negative.

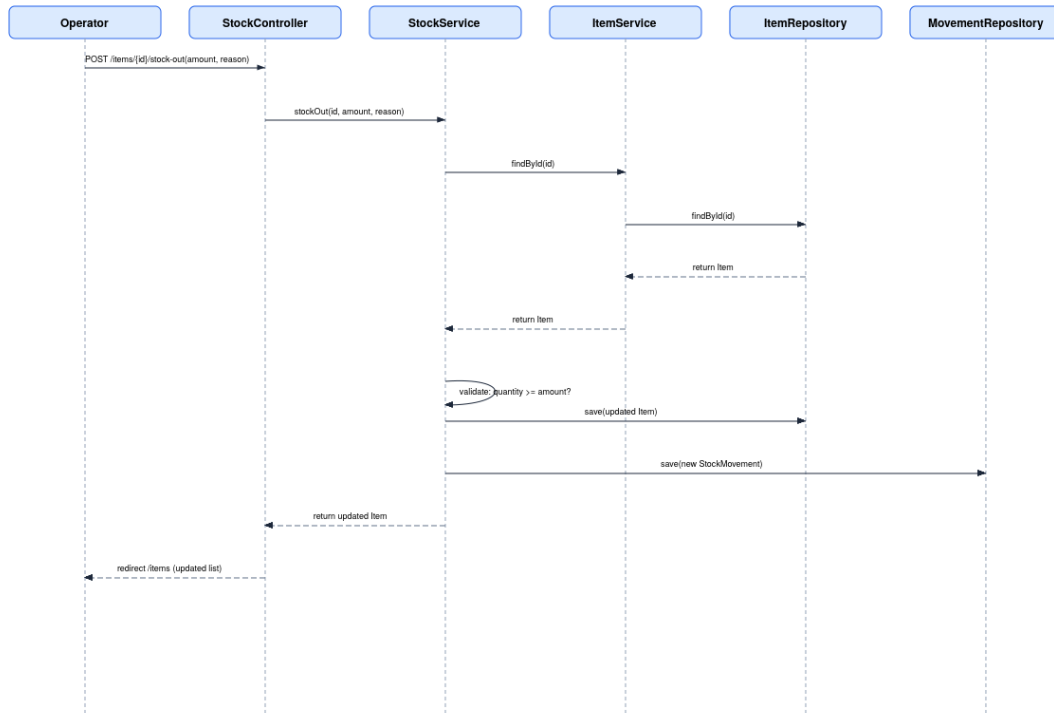


Figure 4: Sequence Diagram (Stock-Out)

7.4 Class / Component Diagram

The class diagram shows the model classes (Item, StockMovement), the repository classes responsible for JSON persistence, the service classes containing business logic, and the controllers that expose the web endpoints.

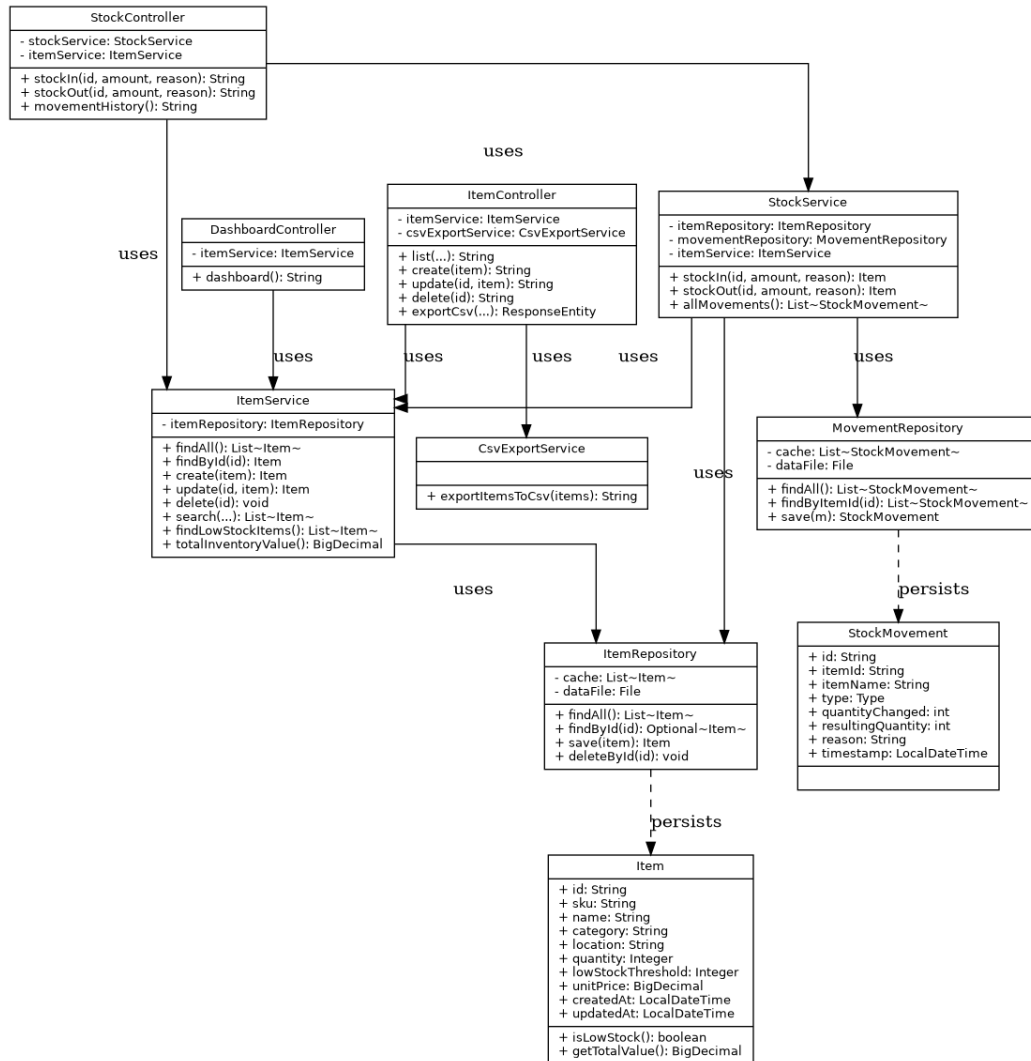


Figure 5: Class / Component Diagram

7.5 Data / Storage Design

Since the application uses file-based storage rather than a relational database, a strict ER diagram does not apply. Instead, the diagram below documents the schema of each JSON file and the logical relationship between them (a StockMovement references the Item it was recorded against via itemId).

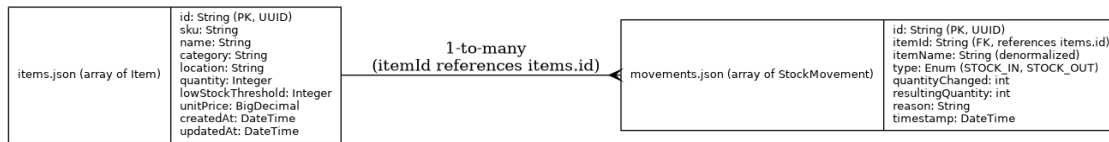


Figure 6: JSON File Schema / Storage Design

8. Design Decisions & Rationale

Decision	Rationale
File-based JSON storage instead of a relational database	Removes the need to install/configure a database server, matching the goal of a lightweight, dependency-free application; Jackson provides robust JSON (de)serialization with minimal code.
Per-item low-stock threshold (not a single global constant)	Real warehouses stock items with very different reorder points (bulk fasteners vs. expensive machine parts), so a global threshold would be inaccurate for most items.
Every quantity change logged as a StockMovement	Provides a full audit trail (a Non-Functional Requirement) and lets the business question 'what happened to this item over time' be answered directly from data rather than inferred.
Layered architecture (Controller / Service / Repository)	Keeps business logic out of controllers and persistence details out of services, so either layer can be changed or unit-tested independently.
Server-rendered Thymeleaf views instead of a separate front-end framework	Matches the chosen Spring Boot web-app scope, keeps the project to a single deployable artifact, and avoids the complexity of a separate JS build pipeline.
ReentrantReadWriteLock around the in-memory cache	Guards against corrupting the cache (and the JSON file) if two requests write at the same time, while still allowing concurrent reads.

9. Implementation Details

9.1 Technology Stack

- Language: Java 17
- Framework: Spring Boot 3.3 (Spring Web, Spring Validation, Thymeleaf)
- Persistence: Jackson ObjectMapper reading/writing JSON files (no JPA/database)
- CSV Export: OpenCSV
- Build tool: Maven
- Testing: JUnit 5 + Mockito

9.2 Package Structure

- model/ — Item, StockMovement (domain classes with business rules like isLowStock())
- repository/ — ItemRepository, MovementRepository (JSON file I/O, guarded by a read-write lock)
- service/ — ItemService, StockService, CsvExportService (all business logic)
- controller/ — DashboardController, ItemController, StockController
- exception/ — ItemNotFoundException, InsufficientStockException, GlobalExceptionHandler

9.3 Key Implementation Notes

- Item.isLowStock() encapsulates the low-stock business rule as a method on the domain object itself, rather than scattering the comparison across controllers/services.
- StockService.stockOut() validates available quantity before mutating state, throwing InsufficientStockException (caught centrally by GlobalExceptionHandler) rather than allowing negative stock.
- ItemRepository and MovementRepository fully encapsulate file I/O — the rest of the application is unaware that persistence is file-based rather than a database, which would make swapping in a real database later a repository-only change.
- Bean Validation annotations (@NotBlank, @Min, @DecimalMin) on the Item model provide first-line input validation, with errors shown inline on the form.

10. Screenshots / Results

[Insert screenshots here after running the application locally: (1) Dashboard showing summary stats and low-stock alerts, (2) Inventory list with search/filter applied, (3) Add/Edit Item form, (4) Movement History page, (5) the Insufficient Stock error page.]

11. Testing Approach

Automated unit tests (JUnit 5 + Mockito) validate the core business rules in isolation from the file-based repositories:

- `ItemTest` — verifies `isLowStock()` at, above, and at the boundary of the threshold, and verifies `getTotalValue()` calculation including the null-safe zero case.
- `StockServiceTest` — mocks `ItemRepository` and `MovementRepository` to verify `stockIn()` increases quantity and logs a movement, `stockOut()` decreases quantity and logs a movement when sufficient stock exists, and `stockOut()` throws `InsufficientStockException` (without logging a movement) when it does not.

Manual end-to-end testing was performed by running the application locally and walking through: adding items, editing and deleting them, performing stock-in and stock-out actions, checking that the dashboard and low-stock badges update correctly, attempting to remove more stock than available (confirming the friendly error page appears), using the search/filter bar, and exporting the filtered list as CSV.

12. Challenges Faced

- Choosing a low-stock model: an early design used one global threshold for all items, but this didn't reflect how different item types are reordered — switching to a per-item `lowStockThreshold` field solved this at the cost of one extra required form field.
- Keeping the Item's quantity and the `StockMovement` audit log consistent: this was solved by ensuring both writes happen inside a single `StockService` method (`stockIn/stockOut`) rather than being triggered from two separate places.
- Concurrent file writes: since all items live in a single JSON file, two simultaneous requests both writing could corrupt the file or silently overwrite each other's changes; this was mitigated with a `ReentrantReadWriteLock` around all repository read/write operations.
- Preventing negative stock: without an explicit check, a stock-out request for more than the available quantity would silently produce a negative number; this was fixed by validating available quantity in `StockService.stockOut()` before mutating state and raising a dedicated `InsufficientStockException`.

13. Learnings & Key Takeaways

- Designing the domain model (`Item`, `StockMovement`) before writing controllers made the business rules (low-stock detection, total value, stock validation) far easier to test and reason about in isolation.
- A layered architecture (`Controller / Service / Repository`) makes it straightforward to unit-test business logic (using Mockito to stub the repository layer) without needing a running web server or real file I/O.
- File-based persistence is a practical option for small, single-user applications, but it also makes clear why real systems move to a database once concurrent access and data volume grow — the manual read-write locking done here is exactly the kind of concern a database would normally handle.
- Centralizing exception handling with `@ControllerAdvice` keeps controllers focused on request/response mapping and avoids repetitive `try/catch` blocks scattered across the codebase.

14. Future Enhancements

- Migrate persistence from JSON files to a relational database (e.g. PostgreSQL via Spring Data JPA) to support larger datasets and true concurrent multi-user access — the Repository layer's interface would not need to change.
- Add user accounts and role-based access control (e.g. 'operator' vs 'admin'), since the current version has no authentication.
- Support multiple warehouses/sites with stock transfer workflows between them.
- Add email/SMS notifications when an item crosses its low-stock threshold, instead of only showing in-app alerts.
- Integrate barcode/QR code scanning for faster stock-in/stock-out on a warehouse floor.
- Add supplier and purchase-order management to close the loop from low-stock alert to reordering.

15. References

- Spring Boot Reference Documentation — <https://docs.spring.io/spring-boot/documentation.html>
- Thymeleaf Documentation — <https://www.thymeleaf.org/documentation.html>
- Jackson Databind (JSON processing) — <https://github.com/FasterXML/jackson-databind>
- OpenCSV Documentation — <http://opencsv.sourceforge.net/>
- JUnit 5 User Guide — <https://junit.org/junit5/docs/current/user-guide/>
- Mockito Documentation — <https://site.mockito.org/>
- Oracle Java 17 Documentation — <https://docs.oracle.com/en/java/javase/17/>